後端 v2 遷移 PRD：food-log 與 sicily-care 搬到 Supabase + Cloudflare

項目	內容
文件版本	v1.0
日期	2026-10-11（台北時間）
產品負責人	Brian（Tzu-Lin Kuo）
執行團隊	Studio：Karina（Lead）、Winter（設計）、Ningning（開發）、Kazuha（驗收）；程式碼由 grok-ship 撰寫
狀態	**已定案**（Brian 2026-10-10／10-11 確認的決定；第 13 節列出仍待決的細節）
依據	`studio/docs/architecture/backend-v2-proposal.md`（Karina，2026-10-10，以下簡稱「提案」）；food-log `docs/PRD.md` v1.5、`docs/kyujin-api.md`；sicily-care `app/docs/spec/`（`decisions.md`、`logging-spec.md`、v1.1–v1.2）

> 標記說明：文中 **（估計）** 表示推估值，不是實測或官方數字。價格與免費額度是提案在 2026-10-10／10-11 查到的官方資料，實作前要再確認一次。

---

1. 背景與問題

1.1 現況

	food-log	sicily-care
前端	React 19 + Vite + Tailwind + shadcn 的 PWA，放在 GitHub Pages	同左
後端	Apps Script `/exec` + Google Sheet	Apps Script `/exec` + Google Sheet，照片放 Drive
驗證	手機上手動貼 `/exec` 網址 + 共享密鑰	同左
寫入	outbox、軟刪除	outbox、軟刪除
Bot	Kyujin：和 App 共用同一個網址與密鑰，自己帶 `client:"kyujin"`	Chaewon：讀 Sheet 和 Drive 做摘要與提醒

1.2 問題

1. **慢**：每次讀寫都要等 Apps Script。Brian 觀察到冷啟動約 **50 秒**；sicily-care 的 `decisions.md` 也記錄閒置後第一次請求常要 20–30 秒，所以前端逾時設成 55 秒。提案在熱的狀態下量到健康檢查要 0.7–1.2 秒，這還沒有讀 Sheet。
2. **設定畫面很怪**：要手動貼 `/exec` 網址和共享密鑰，換手機就要重填。密鑰一旦外流，所有資料都可以被寫入。
3. **沒有真正的身分**：後端只知道密鑰對不對，分不出是 Brian、Mia 還是 Kyujin。`client:"kyujin"` 是 Bot 自己報的，沒有經過驗證。
4. **資料完整性**：Sheet 沒有型別、外鍵和約束，例如提醒時間被自動轉成 `1899-12-30`。
5. **擴充與配額**：每多一個 App，就多一份 Apps Script、Sheet、密鑰和 clasp 部署。Apps Script 有每日執行時間和同時執行數的配額，Bot 和 App 同時打的時候可能要排隊。

2. 目標／非目標

2.1 目標

- **G1**：food-log 和 sicily-care 從 GitHub Pages + Apps Script／Google Sheet 搬到 **Supabase**（Postgres，東京區域，免費方案，一個專案 `studio`），前端改放 **Cloudflare Pages**。
- **G2**：改用 **Google 登入**，拿掉「後端連線設定」畫面（網址 + 共享密鑰）。
- **G3**：權限由資料庫的 **RLS + Gmail 允許清單**控制。food-log 只開放 Brian；sicily-care 開放 Brian 和 Mia。
- **G4**：不再有 Apps Script 冷啟動。前端改成本機優先（IndexedDB + 樂觀寫入 + outbox 同步），按下去的反應不必等後端。
- **G5**：Kyujin 和 Chaewon 改走 **Cloudflare Worker** 的 Bot API，每個 Bot 有自己的窄權限 token，不再和 App 共用密鑰。
- **G6**：每天自動備份和匯出，補上免費方案沒有的自動備份。
- **G7**：遷移期間，**目前的功能開發不能被卡住**。
- **G8**：前端放在 Brian 自己的網域（透過 Cloudflare Registrar 購買），每個 App 用一個子網域。

2.2 非目標

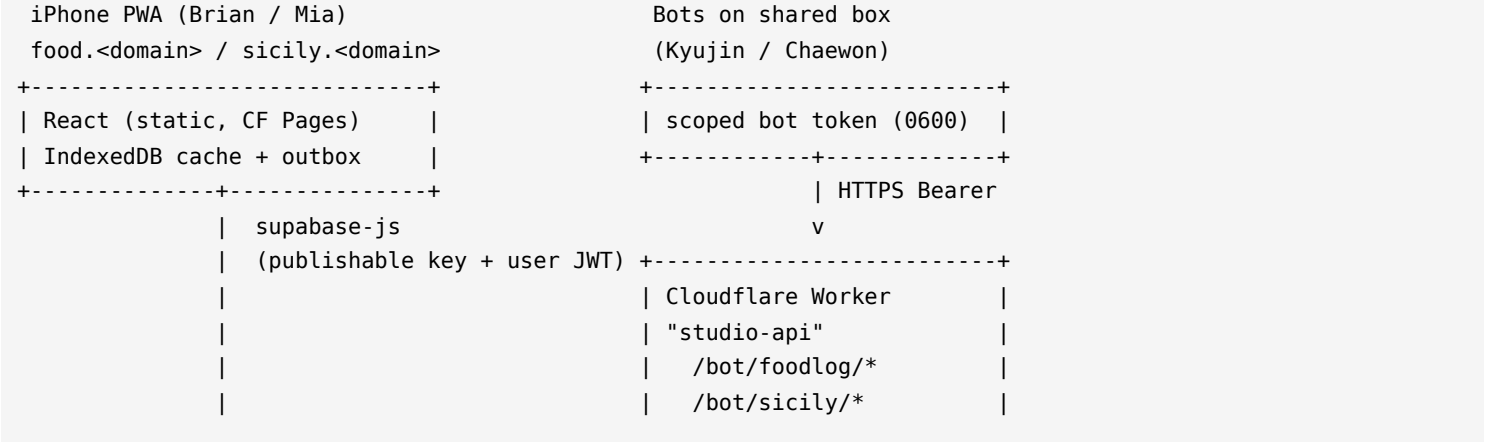
- 不做伺服器端渲染（SSR），也不自己管 VM，**前端維持純靜態**。
- Worker 只用在三件事：Bot API、排程備份／匯出、需要保管機密的邏輯。一般 CRUD 由前端透過 `supabase-js` 直連，靠 RLS 保護，**不另外寫後端 API**。
- 這次不改產品功能、不改 UI 風格（登入畫面與遷移提示除外），也不做推播。
- 不開放 Brian、Mia 以外的人使用，也不做多家庭或多寵物。
- 不升級到付費方案（Supabase Pro、Workers Paid）。只有在第 10 節列的條件出現時，才重新評估。

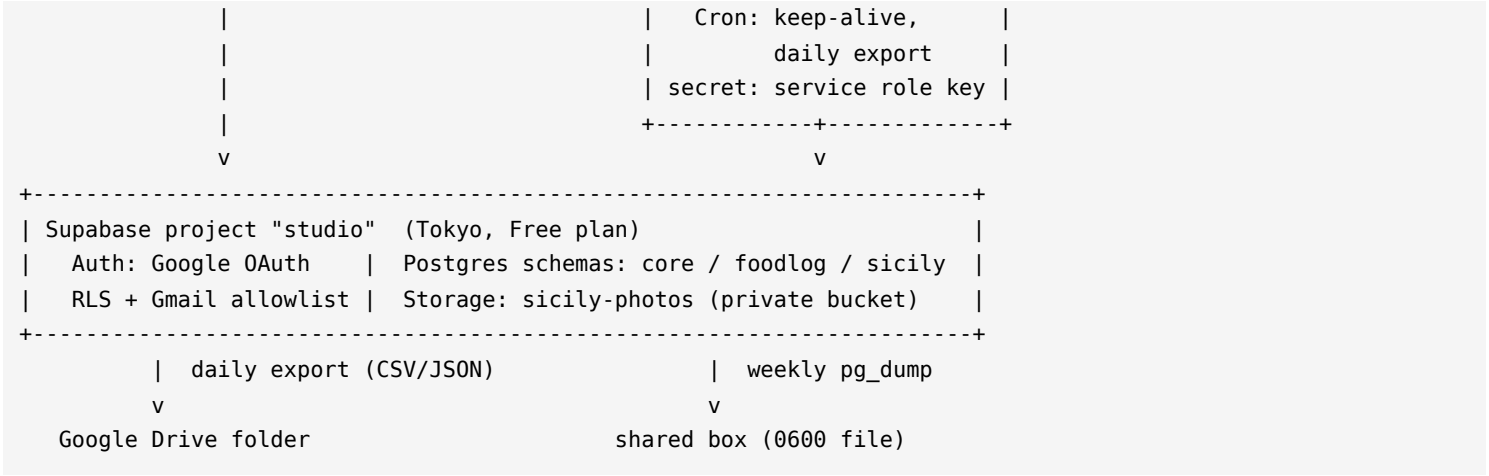
3. 使用者與權限

角色	身分	food-log	sicily-care
Brian	Google 帳號（Gmail，正式地址在實作時填入允許清單）	`owner`：讀寫自己的資料	`member`：讀寫家庭資料
Mia	Google 帳號 `<MIA_GMAIL>`	無權限	`member`：讀寫家庭資料
Kyujin（Bot）	Worker 發的 bot token	限定 scope（見第 7 節）	無權限
Chaewon（Bot）	Worker 發的 bot token	無權限	限定 scope（見第 7 節）
備份工作	Worker Cron（Worker 內部使用 service role）	唯讀匯出	唯讀匯出
其他 Google 帳號	就算登入成功	看不到任何資料	看不到任何資料

- 成員資格存在 `core.app_members(app, user_id, role)`，並且對應到 Gmail 允許清單 `core.allowlist(app, email)`。
- sicily-care 的 `who` 欄位改成伺服器端依 `auth.uid()` 自動填入，不再相信前端傳來的值；畫面上照舊顯示「Brian／Mia」。

4. 架構總覽





4.1 元件與職責

元件	職責	不做的事
Cloudflare Pages	放兩個 App 的靜態檔；每個 PR 有預覽網址；綁定自己網域的子網域	不放任何機密
Supabase `studio`	Postgres（每個 App 一個 schema，另有共用的 `core`）、Google Auth、RLS、Storage	—
Cloudflare Worker `studio-api`	Bot API、Cron（keep-alive、每日匯出）、保管 service role key 等機密	不提供一般使用者的 CRUD
前端同步層	IndexedDB（Dexie）作為畫面唯一的資料來源；outbox 依序推送；依 `updated_at` 游標拉取變更	—

4.2 網域

- 用 Cloudflare Registrar 購買 Brian 自己的網域，DNS 也放在 Cloudflare。
- 子網域建議：`food.<domain>`（food-log）、`sicily.<domain>`（sicily-care）、`api.<domain>`（Worker）。實際名稱由 Brian 決定（見第 13 節）。
- 子網域都要加進 Supabase Auth 的 Redirect URLs 和 Google OAuth 的設定。

4.3 前端本機優先

- **讀**：畫面只讀 IndexedDB。打開 App 時先顯示上次的資料並標示「同步中」；只有第一次使用、本機還沒有資料時，才顯示骨架畫面。這和 food-log v1.5 的載入狀態規則一致。
- **寫**：在同一個 transaction 裡寫入本機表和 outbox → 畫面立刻更新 → 背景推送。用 `upsert … on conflict (id)` 確保冪等。網路錯誤用指數退避重試；遇到 4xx 就把那筆標成失敗，並在 UI 提示使用者處理。
- **拉**：`updated_at > last_pulled_at`（包含已軟刪除的列）。觸發時機：打開 App、回到前景、推送完成後，以及每 60 秒一次。sicily-care 可以加 Supabase Realtime。
- **衝突**：以列為單位，最後寫入的贏（LWW），比較的是伺服器端的 `updated_at`；刪除優先於編輯；Config 以欄位為單位合併。
- 呼叫 `navigator.storage.persist()`，登出時清空 IndexedDB。現有的 outbox 和軟刪除邏輯沿用，只把傳輸層從 GAS `fetch` 換成 `supabase-js`。

5. 資料模型

5.1 共同原則

- 主鍵沿用目前前端產生的字串 id（UUID、`u:`、`s:`、`fm:`、`711:`、`ov:`），outbox 和離線新增都不用改。

- `ts`、`updated_at`、`deleted_at` 用 `timestamptz`；`day` 用 `date`（台北日期）。
- 軟刪除用 `deleted_at`，取代原本的 `deleted` 布林欄位。
- 每張表都有 `user_id`（food-log）或 `household_id`（sicily-care；目前只有一個家庭，先保留這個欄位方便擴充），以及 `created_by`（`app`／`kyujin`／`chaewon`）。
- `updated_at` 由伺服器端的 trigger 設定，作為同步游標，不採用手機的時鐘。
- schema 要加進 Supabase API 的 exposed schemas；前端用 `supabase.schema('foodlog')`。
- 所有 migration 都放在 git 的 `supabase/migrations/`，資料庫可以從空的狀態重建。

5.2 `core` schema

表	欄位	用途
`core.app_members`	`app, user_id, role`	App 成員：food-log 只有 Brian（`owner`）；sicily-care 有 Brian 和 Mia（`member`）
`core.allowlist`	`app, email`	Gmail 允許清單。使用者登入後，以 email 對應到 `app_members`
`core.bot_tokens`	`id, bot, scopes text[], token_hash, created_at, revoked_at`	Bot token。資料庫只存 hash

5.3 food-log（`foodlog` schema）

原 Sheet	Postgres 表	主要欄位	備註
Items	`foodlog.items`	`id text pk, user_id uuid null, kind, base_id, src check in ('7-11','全家','其他'), name, serving, serving_g numeric, kcal, protein, fat, carb numeric, fav bool, created_by, store_id null, ts, updated_at, deleted_at`	官方品項（`fm:`、`711:`）的 `user_id` 是 null，所有人都能讀，只有批次工作能寫。`ov:`（補填營養素）和 `u:`（自建）屬於使用者
Stores	`foodlog.stores`	`id text pk, user_id, name, created_by, ts, last_eaten_at, updated_at, deleted_at`	給 v1.5 的「常吃店家」。店家品項就是 `items.store_id` 有值的品項
Logs	`foodlog.logs`	`id text pk, user_id, day date, item_id null → items, store_id null, name, src, qty, grams, kcal, protein, fat, carb, source check in ('app','quick','kyujin'), created_by, ts, updated_at, deleted_at`	營養素是記錄當下的快照。`source`、`store_id` 依 `kyujin-api.md` 和 PRD v1.5 保留
Weights	`foodlog.weights`	`user_id, day date, kg numeric, id, ts, updated_at, deleted_at，pk (user_id, day)`	一天一筆，用 upsert。用 check 擋掉未來日期（台北時間），以及 kg 超出 30.0–250.0 的值
Config	`foodlog.config`	`user_id pk, target_kcal, target_protein, target_fat, target_carb, tdee null, remind_meal_times time[], remind_weight_time time null, updated_at`	從 key/value 改成有型別的欄位，修掉時間被轉成日期的 bug。`fav_ids` 改用 `items.fav`
catalog-v1.json	繼續當靜態檔放在 Pages	—	2,574 項官方品項。必要時同步一份到 `items`，讓 SQL 加總和 Bot 查得到

- `summary`、`trends` 改成 SQL RPC：`foodlog.day_summary(day)`、`foodlog.week_summary(week_start)`。App 和 Kyujin 共用同一份計算，取代現在 GAS 和前端各寫一份的狀況。

5.4 sicily-care（`sicily` schema）

原 Sheet	Postgres 表	備註
Feed	`sicily.feed(id, ts, who uuid, food_id → foods, food_name, qty, unit, grams_est, eaten_pct, reaction, note, …)`	who 改成使用者 uuid
Litter	`sicily.litter(…, all_normal bool, urine_count int, urine_size, urine_flags text[], stool_count int, stool_cat, purina_range, stool_amount, stool_flags text[], note)`	逗號分隔的字串改成 `text[]`
Weight	`sicily.weight(…, kg, method, note)`	
Med	`sicily.med(…, kind, product, dose, next_due date, note)`	
Issue	`sicily.issue(…, category, sub, severity, note, resolved bool)`	`photo_ids`／`photo_urls` 移到 `sicily.photos`
Care	`sicily.care(…, kind, note)`	v1.1 新增的週期維護
Foods	`sicily.foods(food_id pk, name, brand, kind, unit, default_qty, grams_per_unit, fav, active)`	
Config	`sicily.config`：key/value 或有型別的欄位	實作時依實際的 key 決定（例如 `litter_clumping`）
Drive 照片	`sicily.photos(id, entry_table, entry_id, storage_path, w, h, taken_at, uploaded_by)` + 私有 bucket `sicily-photos`	用有時效的 signed URL 顯示；上傳前在手機端壓縮

- 共同欄位：`household_id`、`ts`、`updated_at`、`deleted_at`、`created_by`。

6. 認證與安全

6.1 使用者登入

- 使用 Supabase Auth + Google OAuth。PWA 第一次開啟時按「用 Google 登入」，之後 session 自動續期。**不再需要輸入 `/exec` 網址或密鑰。**
- iOS 加到主畫面的 PWA 有自己的儲存空間，所以登入要在 PWA 裡做一次。OAuth 會跳到 Safari 再轉回來，這一點要在真機上驗證。
- Google OAuth 同意畫面先維持 **Testing** 狀態，把 Brian 和 Mia 加為 test users，不用送 Google 審核。Testing 狀態的 refresh token 可能 7 天就過期，要在真機上確認；必要時改成 In production。
- 關掉陌生人註冊，或在註冊／登入時檢查允許清單。

6.2 RLS 與允許清單

- **所有表預設拒絕**：啟用 RLS，沒有對應 policy 就讀寫不了。
- food-log：`user_id = auth.uid()`，而且 `core.is_member('foodlog', auth.uid())`。官方品項（`user_id is null`）對成員唯讀。
- sicily-care：`core.is_member('sicily', auth.uid())` 的人可以讀寫同一個家庭的資料。
- `core.is_member()` 會同時檢查 `app_members`，以及使用者 JWT 裡的 email 是否在 `core.allowlist`。不在清單上的帳號，就算登入成功也查不到任何資料。
- Storage bucket `sicily-photos` 設為私有，policy 和 `sicily` schema 相同。

示意：

```
create policy own_rows on foodlog.logs for all
  using (user_id = auth.uid() and core.is_member('foodlog', auth.uid()))
  with check (user_id = auth.uid() and core.is_member('foodlog', auth.uid()));

create policy household on sicily.feed for all
  using (core.is_member('sicily', auth.uid()))
  with check (core.is_member('sicily', auth.uid()));
```

6.3 Bot 存取（經由 Worker，使用限定 scope 的 token）

- Bot **不會直接拿到 service role key**。這把 key 會繞過所有 RLS，所以只放在 Worker 的 secret 裡。
- 每個 Bot 有一把自己的隨機 token（32 bytes），資料庫只存 hash。Worker 的處理流程：驗證 hash → 檢查 `revoked_at` → 檢查 scope → 用 service role 執行**固定的查詢**（不接受任意 SQL）→ 在 `created_by` 填入 Bot 名稱，留下稽核紀錄。
- token 放在共用電腦的 `~/.config/studio/bots/<bot>.env`（權限 600）。**不進 git、不進前端、不貼在聊天裡。**
- 撤銷：設定 `revoked_at`，立即失效。輪替：先發一把新的，換上之後再撤銷舊的。

6.4 機密放在哪裡

機密	放在哪裡	誰看得到
Supabase URL、publishable（anon）key	前端 build 時的環境變數	公開，安全性靠 RLS
Supabase service role key	只放 Worker secret（`wrangler secret put`）	只有 Worker
Google OAuth client secret	只放在 Supabase Auth 設定	Brian
Bot token	共用電腦上的 600 權限檔；資料庫只存 hash	對應的 Bot
資料庫密碼／連線字串	Brian 的密碼管理器；備份用的連線字串放 Worker secret，或共用電腦上的 600 權限檔	Brian、備份工作
Cloudflare API token	只限 Pages + Workers 權限，用於部署	Ningning（部署）

- Brian 一律透過**安全輸入**交付 service key 和 API token，不在群組或聊天裡貼。
- 如果用 CI 部署，只放 Cloudflare 的部署 token，不放 service role key。

7. Bot API

- Base URL：`https://api.<domain>/bot/...`（還沒綁網域之前用 `studio-api.<account>.workers.dev`）。
- 驗證方式：`Authorization: Bearer <token>`。請求和回應都是 JSON。
- 錯誤碼沿用現有的：`forbidden`、`bad_record`、`duplicate_name`，另外新增 `unauthorized`（token 無效或已撤銷）。
- 原則：**對外介面盡量和現在的 `kyujin-api.md` 一樣**，Bot 端只需要換網址和 token。文件跟著 cutover 一起更新。

7.1 Kyujin（food-log）

Endpoint	說明	Scope
`GET /bot/foodlog/read`	Items、Stores、Config 全部給（含已刪除）；Logs 往回 35 天；Weights 往回 90 天	`foodlog:read`
`GET /bot/foodlog/summary?day=`	當天的 totals、targets、remaining（可以是負數），由 `day_summary` 計算	`foodlog:read`

Endpoint	說明	Scope
`GET /bot/foodlog/trends?weekStart=`	週摘要（週一開始），由 `week_summary` 計算，和 App 一致	`foodlog:read`
`GET /bot/foodlog/history`	歷史紀錄（沿用現有的 `history` 語意）	`foodlog:read`
`POST /bot/foodlog/stores`	upsertStore：新建時 `created_by=kyujin`；可以改名；不能刪除或復原	`foodlog:stores.write`
`POST /bot/foodlog/items`	upsertItem：Kyujin 自建的 `u:` 品項，以及店家品項；不能改官方品項或 `ov:`	`foodlog:items.write`
`POST /bot/foodlog/logs` 及 `PATCH`、`DELETE`	只能新增、修改、軟刪除、復原 `source=kyujin` 的 Logs；不能改 `source`	`foodlog:logs.quick_write`
`POST /bot/foodlog/weights`	upsertWeight：同一天覆寫；kg 30.0–250.0；不能是未來日期	`foodlog:weights.write`

- Kyujin **不可以**：`setConfig`、動 `source=app／quick` 的 Logs、改沒有店家的 App 自建品項、補填官方品項。這些規則和 `kyujin-api.md` 相同，在 Worker 裡判斷，並用 RLS 與 check 雙重把關。

7.2 Chaewon（sicily-care）

Endpoint	說明	Scope
`GET /bot/sicily/read`	讀各表近期資料，供摘要與提醒用（例如 36 小時沒清砂、尿塊量異常、連續軟便）	`sicily:read`
`GET /bot/sicily/photos/:id`	回傳照片的短效 signed URL	`sicily:read`
寫入類 endpoint	**待決**：依 Chaewon 現在的權限，決定是唯讀還是可以寫	`sicily:write`（如果開放）

8. 遷移計畫

8.1 原則

- **不卡住功能開發**：food-log 的「常吃店家」、載入狀態等工作照原計畫在現有架構上進行。遷移在獨立分支開發，新的同步層只取代傳輸層，所以這段期間做的 UI 不會白費。只有切換當晚暫停部署新功能。
- 先搬 food-log（資料少、還在早期），穩定兩週後再搬 sicily-care。
- 每個階段都要能回滾；舊系統至少保留 30 天。

8.2 階段

階段	內容	負責	工作量
0. 前置作業	Brian 建立帳號、買網域、設定 OAuth、交付金鑰、量冷啟動基準（見第 12 節）	Brian	約 30 分鐘（估計，不含網域與 DNS 生效時間）
1a. 平台	schema、RLS 與 RLS 自動化測試、migration 流程、Worker 骨架（bot token、keep-alive、備份 Cron）、Pages 與子網域	grok-ship；Ningning 檢查並部署	grok-ship × 1
1b. food-log 前端	IndexedDB 本機優先同步層、Google 登入、拿掉設定畫面；Sheet 匯入腳本	grok-ship；Winter 負責登入畫面與遷移提示的設計	grok-ship × 1

階段	內容	負責	工作量
1c. food-log 切換	Kyujin API 遷移、cutover、回滾腳本；接著觀察兩週	grok-ship；Kazuha 驗收	grok-ship × 1
2a. sicily-care 平台	schema、同步層、Mia 的權限	grok-ship	grok-ship × 1
2b. sicily-care 切換	照片遷移、Chaewon API、cutover	grok-ship；Kazuha 驗收	grok-ship × 1
緩衝	真機問題、OAuth 細節	—	grok-ship × 0–1
3. 收尾	舊系統保留至少 30 天後，關閉 Apps Script 寫入、封存 Sheet	Brian 確認	—

合計 **5–6 次 grok-ship**（提案的估計）。

8.3 從 Sheet 匯入資料

1. 用現有的 `read`／`history` 或直接匯出 CSV，取得各分頁的資料。
2. 轉換腳本：`deleted` 轉成 `deleted_at`、修正時間格式（例如 `1899-12-30` 的提醒時間）、補上 Brian 的 `user_id`、sicily-care 的 `who` 文字轉成兩人的 uuid、逗號字串轉成陣列。
3. 匯入時用 upsert，**腳本可以重跑（冪等）**。
4. 比對：每張表的列數，以及 food-log 每天的熱量與營養素加總、sicily-care 每天各類紀錄的筆數。
5. sicily-care 照片：從 Drive 下載 → 上傳到 `sicily-photos` → 寫入 `sicily.photos`，比對張數和檔案大小。**Drive 上的原檔保留，不刪除。**

8.4 雙軌運行與切換

1. **雙軌期**：舊系統照常是正式環境。新系統部署在子網域，資料是匯入的副本，給 Ningning 和 Kazuha 驗證、Brian 試用（包括 iOS PWA 的 Google 登入）。這段期間可以重複跑匯入。
2. **切換當晚**（food-log 由 Brian 選時間；sicily-care 配合 Mia 的作息）：
   - 把 GAS 設成**唯讀**（寫入回傳 `read_only`）→ 確認舊 App 的 outbox 已經清空 → 最後再匯入一次差異 → 驗證比對結果。
   - 把 Bot 切到新的 Worker 和 token，舊的共享密鑰不再給 Bot 使用。
   - Brian（以及 sicily-care 的 Mia）打開新網址，用 Google 登入，重新加到主畫面，確認今天的資料都在。
   - GitHub Pages 的舊版改成提示頁，導向新網址。
3. 切換後觀察兩週，每天都要有備份成功。

8.5 回滾

- 觸發條件：資料遺失或錯誤、無法登入、同步異常，而且當下修不好。
- 步驟：把 GAS 改回可寫 → 用反向匯出腳本，把切換後新增或修改的紀錄寫回 Sheet（這支腳本在階段 1 就要寫好、測好）→ 舊前端恢復成主要網址 → Bot 換回舊設定。
- 目標：**30 分鐘內**回到舊系統，最多少掉幾筆、可以手動補回（提案的目標）。
- 舊的 Apps Script 部署和 Sheet 在切換後**至少保留 30 天**。

9. 備份

項目	頻率	位置	內容
每日匯出	每天 04:00（台北時間），由 Worker Cron 執行	Google Drive 資料夾，或一份唯讀 Google Sheet（待決）	各表 CSV／JSON，Brian 也可以拿來看原始資料、做樞紐分析

項目	頻率	位置	內容
每週 pg_dump	每週	共用電腦（600 權限）	完整資料庫
照片	保留 Drive 原檔；新照片只在 Supabase Storage	—	新照片的備份方式待決
keep-alive	每天，由 Worker Cron 執行	—	每天對資料庫做一次真正的查詢，避免免費專案「一週不活動就暫停」

- 驗證：每季至少做一次「從備份還原到空的資料庫」演練（建議頻率），第一次在 food-log 切換前完成。
- Worker 免費方案最多 5 個 Cron Trigger，排程數量要控制在這個範圍內。
- 從 Worker 寫入 Drive 需要 Google 憑證（例如 service account），這也屬於 Worker 保管的機密。

10. 成本

項目	方案	每月費用
Supabase	Free：資料庫 500 MB、Storage 1 GB、egress 5 GB、50,000 MAU、最多 2 個啟用中的專案；**沒有自動備份；一週不活動會暫停**	$0
Cloudflare Pages	靜態資源請求免費、不限量	$0
Cloudflare Workers	Free：每天 10 萬次請求、每次 10 ms CPU、5 個 Cron Trigger	$0
Google OAuth	—	$0
網域	Cloudflare Registrar	依 TLD 而定；提案以 .app 為例，約 $1／月，年繳（估計）
合計		**網域費 + $0**

- **用量（估計）**：food-log 每天約 10–20 筆，一年不到 1 萬列，約 5–10 MB；sicily-care 的照片每天 1–2 張、每張壓到約 300 KB，一年約 150–200 MB。照這個速度，1 GB Storage 大約可以撐 4–5 年（估計）。
- **重新評估升級的條件**：專案被暫停造成困擾、照片接近 1 GB，或需要官方每日備份。這時再考慮 Supabase Pro，約 $25／月（以官網為準），含 7 天每日備份，而且不會暫停。

11. 驗收標準

由 Kazuha 代表 Brian 驗收，每一項都要有證據（測試結果、截圖或比對報表）。

登入與權限

1. 兩個 App 都已經沒有「後端連線設定」畫面，第一次開啟只需要用 Google 登入。在 iPhone 加到主畫面的 PWA 裡，可以完成登入並保持登入（過期行為要記錄下來）。
2. RLS 自動化測試全部通過：Brian 可以讀寫 food-log；Mia 讀寫 food-log 得到 0 列或被拒；Brian 和 Mia 都可以讀寫 sicily-care；不在允許清單上的 Google 帳號，在所有表都讀不到資料、也寫不進去；匿名請求一律被拒。
3. 前端 bundle 和 git 裡**沒有** service role key、bot token、資料庫密碼（自動掃描通過）。
4. sicily-care 的 who 由伺服器端填入，前端偽造的值會被忽略。

效能與離線

5. 新增一筆紀錄時，畫面在 100 ms 內更新（提案的目標），不必等網路；離線時也能記錄，上線後自動補送，不會重複，也不會遺失（e2e 測試涵蓋「離線 → 上線」）。
6. 冷啟動時，任何時間點都不會出現和真實資料不一致的數字（food-log v1.5 的驗收標準）。
7. 以 Brian 量到的 Apps Script 冷啟動作為基準，新系統打開 App 時，不必等後端就能看到上次的資料。

資料遷移

8. 每張表的列數和舊 Sheet 相同（排除已知、有記錄的無效列）；food-log 每天的熱量與營養素加總和 Sheet 一致；sicily-care 照片的張數和檔案大小都一致。
9. 原本的提醒時間（Config）轉換正確，沒有 `1899-12-30` 這類錯誤。

Bot

10. Kyujin 回答「今天還剩多少」的數字和 App 一致；`kyujin-api.md` 裡的允許和禁止案例，在新 API 上的結果都相同。
11. 撤銷 token 後，下一個請求就回傳 `unauthorized`；Kyujin 的 token 呼叫 `/bot/sicily/*` 會被拒絕。
12. Bot 寫入的列，`created_by` 都是 Bot 的名稱。

營運

13. keep-alive 和每日匯出連續 14 天成功；至少完成一次從備份還原的演練。
14. 回滾演練（在雙軌期進行）可以在 30 分鐘內回到舊系統。
15. 舊系統在切換後保留至少 30 天，期間是唯讀。
16. 遷移期間，food-log 原本排定的功能（常吃店家、載入狀態）照常上線，沒有被遷移卡住。

12. Brian 的前置作業

#	事項	說明	為什麼一定要本人做
1	建立 Supabase 帳號和專案 `studio`	區域選 Northeast Asia (Tokyo)、方案選 Free；資料庫密碼存進密碼管理器	帳號擁有者
2	建立 Cloudflare 帳號	產生一支只限 Pages + Workers 的 API token	帳號擁有者
3	購買網域	透過 Cloudflare Registrar 購買，並決定子網域名稱	付款、擁有權
4	Google OAuth client	在 Google Cloud Console 建立 Web application 類型的 client，設定同意畫面（Testing），把 Brian 和 Mia 加為 test users；把 Supabase 的 callback URL 和各子網域填進去，再把 client id／secret 填回 Supabase Auth	需要 Brian 的 Google Cloud 帳號
5	交付金鑰	透過**安全輸入**提供 Supabase service role key、資料庫連線字串和 Cloudflare API token，不貼在聊天裡	機密
6	量冷啟動基準	在閒置一段時間後（例如早上第一次開 App），量健康檢查和實際 `read` 的時間，作為改善前的基準	需要真機和密鑰
7	通知 Mia	在 sicily-care 切換前約 2 週告訴 Mia：之後要改用 Google 登入、會換新網址、要重新加到主畫面；切換當天陪她第一次登入	對 Mia 的溝通
8	切換當天驗證	在手機上確認資料都在，並重新加到主畫面	真機驗證

- 免費方案不需要綁信用卡。網域需要付款方式。

13. 風險與待決問題

13.1 風險

風險	影響	緩解方式
Supabase 免費專案一週不活動就會暫停	出國或一陣子沒記錄時，App 會連不上	Worker Cron 每天做 keep-alive 查詢；就算被暫停，也可以在 Supabase Studio 還原，而且本機資料還在
免費方案沒有自動備份	資料遺失	每日匯出 + 每週 `pg_dump` + 還原演練
iOS PWA 的 OAuth 跳轉，以及 Testing 狀態的 refresh token 期限	登入卡住，或常常要重新登入	階段 1b 先在真機驗證；必要時把同意畫面改成 In production
RLS 寫錯	資料外洩或寫不進去	預設全部拒絕；每個角色都有讀寫的自動化測試
同步層的 bug（重複或遺失）	資料不一致	冪等 upsert、前端產生 id、離線 → 上線的 e2e 測試
遷移時資料轉換錯誤	歷史資料不正確	比對列數和每日加總；舊 Sheet 保留
所有 App 共用一個專案	一出事就全部受影響	以目前的規模可以接受；本機優先讓 App 離線時仍能使用
Mia 不習慣新的登入方式	sicily-care 有幾天沒有紀錄	提前約 2 週通知、配合她的作息切換、第一次登入有人陪
遷移和功能開發同時進行，產生衝突	功能延誤	分支隔離、只替換傳輸層、切換當晚才暫停部署
免費額度或價格變動	成本或可用性改變	實作前重新查證；資料是標準 Postgres，可以用 `pg_dump` 帶走

13.2 待決問題

1. **子網域名稱**和要買的網域（例如 `food.<domain>`、`sicily.<domain>`、`api.<domain>`）。
2. **Chaewon 的權限**：只能讀，還是也可以寫？可以寫哪些表？
3. **每日匯出的格式**：寫進唯讀的 Google Sheet，還是 CSV／JSON 放在 Drive 就好？Worker 寫 Drive 要用哪一種 Google 憑證？
4. **舊系統保留多久**：30 天後就關閉 Apps Script 嗎？
5. **sicily-care 切換的日期**：food-log 穩定兩週後，要配合 Mia 方便的時間。
6. **catalog-v1.json** 要不要同步進 `foodlog.items`（讓 SQL 和 Bot 查得到）？
7. **sicily-care 新照片的備份方式**：每日匯出要不要包含 Storage 裡的照片？
8. **Brian 的 Gmail 地址**，以及 sicily-care 允許清單最終的名單（目前是 Brian 和 `<MIA_GMAIL>`）。